SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

TOTAL:94/100

COURSE CODE: CSA4303

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact. 3. Determine whether the status code matches the response correctness. 4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method URL Example Data Visibility Security Level GET /login?user=admin&pass=1234 Visible in URL Low POST /login Hidden in body Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why. 3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
```

```
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

*I **Nirosha.M(192311411)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any*

violation of academic integrity rules will result in disciplinary action.

Signature of the student: Nirosha.M

RUBRICS FOR CALCULATION:

Criteria	**Weigh tage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Understanding of Data	24	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	24	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	19	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data

Conclusion	19	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	9	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

TOTAL:94/100

1: **HTTP Response Analysis**

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

Given HTTP response header:

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

1.1 Identify any incorrect or invalid fields in the header.

The following issues are identified:

- Content-Length: -150: Invalid because the `Content-Length` header must contain a non-negative decimal integer representing the response body size in bytes.

- Date: The weekday and calendar date do not match. April 1, 2026, is a Wednesday, not a Monday. ● HTTP/1.1 200 OK: This status line is valid, but its correctness depends on whether the request was successfully processed.

The `Content-Type: text/html` and `Connection: keep-alive` headers are valid in this context. **1.2 Analyze the issue with Content-Length and explain its impact.**

The `Content-Length` header specifies the size of the HTTP response body in bytes. In the given response, `Content-Length: -150` is invalid because negative values are not permitted. Its possible impacts include:

1. The response may be rejected or handled as a malformed message by an HTTP implementation.
2. The browser or client may fail to interpret the response correctly.
3. If the message length is incorrectly specified with a valid but inaccurate value, the client may wait for additional data or treat subsequent bytes as another message.
4. Intermediaries such as proxies may experience message-framing problems.

Correction: Set `Content-Length` to the actual non-negative number of bytes in the response body. Alternatively, HTTP/1.1 can use valid chunked transfer encoding when appropriate.

1.3 Determine whether the status code matches the response correctness.

The status code `200 OK` indicates that the request was successfully processed.

However, the response contains an invalid `Content-Length` value. Therefore, the status code alone does not guarantee that the entire response is correctly formatted.

- If the server successfully processed the request but generated a malformed response header, `200 OK` may describe the application-level result, but the response is still invalid.
- If the server cannot fulfill the request, it should return an appropriate error status, such as `400 Bad Request` for an invalid client request or `500 Internal Server Error` for an unexpected server-side failure.

The appropriate status code depends on the actual outcome; the invalid header should be corrected independently.

1.4 Suggest corrections for the identified errors.

A corrected example is:

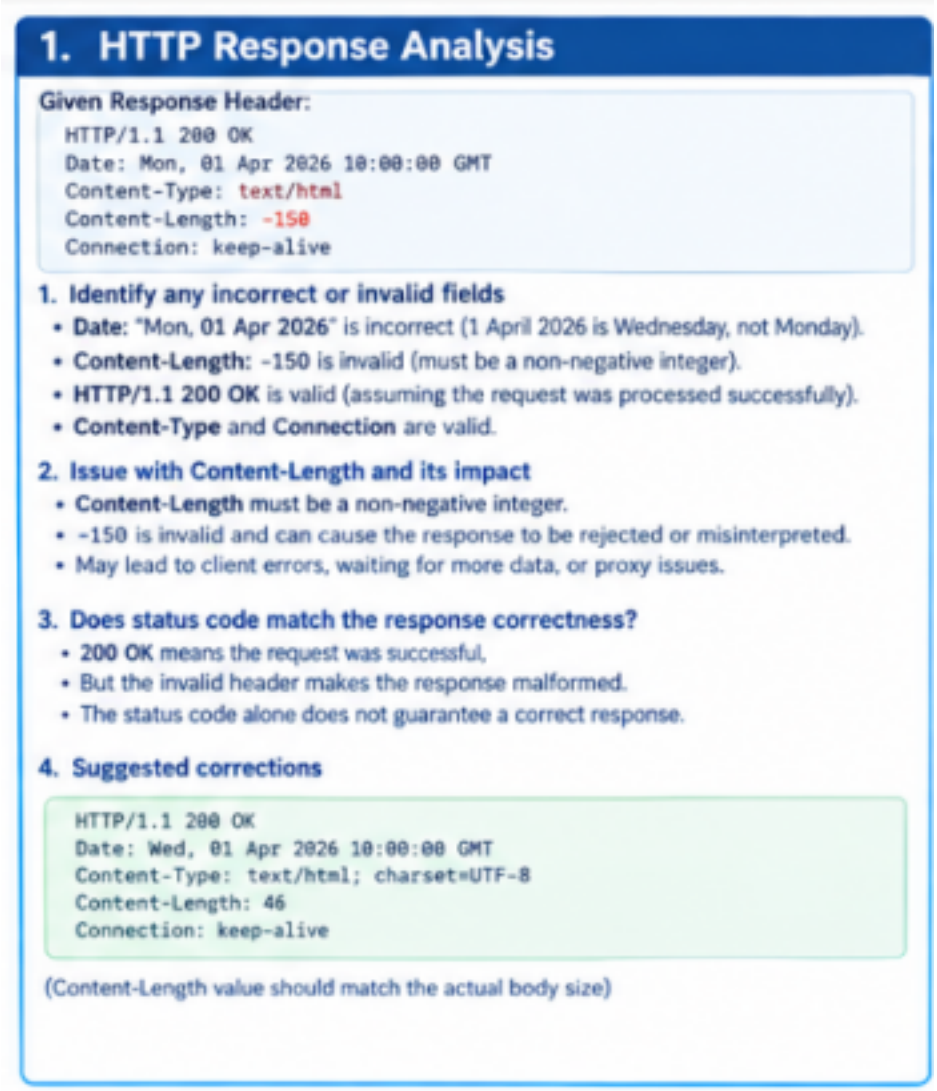
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 46
Connection: keep-alive

<html><body><h1>Hello World</h1></body></html>

The example assumes that the response body is exactly 46 bytes when encoded as UTF-8. In production, the server should calculate the actual byte length rather than hard-code it.

The corrections are:

1. Replace the negative `Content-Length` with the actual response body size.
2. Correct the weekday in the `Date` header.
3. Specify the character encoding in the content type when appropriate.
4. Ensure that the status code accurately reflects the result of processing the request.
5. Use valid HTTP message framing and follow the HTTP specification.



1. HTTP Response Analysis

Given Response Header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

1. Identify any incorrect or invalid fields

- **Date:** "Mon, 01 Apr 2026" is incorrect (1 April 2026 is Wednesday, not Monday).
- **Content-Length:** -150 is invalid (must be a non-negative integer).
- **HTTP/1.1 200 OK** is valid (assuming the request was processed successfully).
- **Content-Type** and **Connection** are valid.

2. Issue with Content-Length and its impact

- **Content-Length** must be a non-negative integer.
- -150 is invalid and can cause the response to be rejected or misinterpreted.
- May lead to client errors, waiting for more data, or proxy issues.

3. Does status code match the response correctness?

- **200 OK** means the request was successful,
- But the invalid header makes the response malformed.
- The status code alone does not guarantee a correct response.

4. Suggested corrections

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 46
Connection: keep-alive
```

(Content-Length value should match the actual body size)

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.

2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

Given DOM representation:

```
<html>

<head>

<title>TestPage</title>

</head>
<body>

<div>

<p>HelloWorld

</div>

</body>

</html>
```

2.1 Identify missing or improperly closed elements.

The main issue is that the paragraph element `<p>` is not explicitly closed with `</p>` before the closing `</div>`. The document also omits the HTML5 doctype declaration.

Although the browser can often recover from these issues, the source should be corrected to follow HTML standards.

2.2 Analyze how the browser will interpret this structure.

Modern browsers use HTML parsing rules to recover from many markup errors.

In this example:

1. The browser encounters the opening `<p>` element inside the `<div>`.
2. When it encounters `</div>`, the HTML parser implicitly closes the paragraph if necessary to maintain a valid DOM structure.

3. The browser constructs a DOM tree that may differ from the source structure the developer intended.
4. The page may still display, even though the HTML source is incomplete.

Browsers do not necessarily stop rendering when an element is missing its closing tag. However, relying on automatic recovery can make debugging and maintenance difficult.

2.3 Determine the impact on rendering and layout.

The malformed structure can cause the following issues:

- Unexpected paragraph spacing due to default CSS margins.
- Incorrect placement of later elements if the parser recovers differently from the developer's expectations.
- Unexpected results when CSS selectors depend on the DOM hierarchy.
- Reduced source-code readability and maintainability.
- Potential confusion for developers and accessibility tools when the document structure becomes more complex.

For this simple example, the browser will likely render `HelloWorld` normally. The major concern is structural correctness and the potential impact on more complex pages.

2.4 Suggest corrections to make the DOM valid.

Corrected HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>TestPage</title>
</head>
<body>
    <main>
        <div>
            <p>HelloWorld</p>
```

```
    </div>
  </main>

</body>

</html>
```

Explanation: The corrected code adds the doctype, language, character encoding, and viewport metadata. It also closes the paragraph explicitly and uses `<main>` to identify the primary page content. These changes improve standards compliance, readability, and maintainability.



2. DOM Structure Analysis

Given DOM Tree (with issues):

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

1. Missing or improperly closed elements

- <p> tag is not explicitly closed.
- Missing <!DOCTYPE html> declaration.

2. How browser interprets it

- The browser auto-closes the <p> when it encounters .
- It builds a correct DOM tree using error recovery.
- The page may still render, but structure differs from the source.

3. Impact on rendering and layout

- Unexpected spacing due to default styles.
- CSS selectors may behave differently.
- Harder to maintain and debug.
- Accessibility tools may get confused.

4. Corrected Version

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>TestPage</title>
  </head>
  <body>
    <main>
      <div>
        <p>HelloWorld</p>
      </div>
    </main>
  </body>
</html>
```

3: GET vs POST in Login System

A login system uses two different methods:

Method URL Example Data Visibility Security Level GET /login?user=admin&pass=1234 Visible in URL Low POST /login Hidden in body Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why. 3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

Given comparison:

Feature	GET	POST
Example	`/login?user=admin&pass=1`	`23 4 /login`
Data transmission	Usually appended to URL as query parameters	Usually placed in the request body
Visibility	Parameters can appear in the URL	Data is not normally displayed in the URL
Browser history	Query parameters may be stored	Request body is not normally stored in browser history
Security	Unsuitable for passwords	More suitable for submitting credentials
Encryption	Not provided by GET itself	Not provided by POST itself
Recommended use	Retrieving resources, searches	Login forms and data submission

3.1 Interpret the differences in data transmission between GET and POST. GET method:

GET sends form data as query parameters in the URL.

Example:

GET /login?user=admin&pass=1234 HTTP/1.1
Host: example.com

The parameters can be visible in browser history, copied URLs, server logs, and monitoring systems. GET requests are generally used to retrieve information.

POST method:
POST normally sends form data in the request body.

Example:

POST /login HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

user=admin&pass=1234

The data is not included in the URL. POST is suitable for submitting credentials and other form data, but it does not automatically encrypt the request body.

3.2 Analyze which method is more suitable for login and why.

The POST method over HTTPS is more suitable for login authentication.

Reasons:

1. Passwords are transmitted in the request body instead of appearing in the URL.
2. Credentials are less likely to be exposed through browser history and URL sharing.
3. POST is appropriate for submitting authentication data to the server.
4. HTTPS encrypts the request and response during transmission.
5. The server can validate the credentials and establish a secure authenticated session.

Therefore, a login form should use POST with HTTPS and proper server-side authentication. **3.3 Identify security risks in using GET for authentication.**

Using GET for authentication creates several security risks:

● URL exposure: Passwords may appear in browser history, copied links, and screenshots. ● Server logs: Web servers, proxies, and monitoring systems may record query parameters. ● Referrer leakage: URLs containing credentials may be disclosed through referrer information in some circumstances.
● Caching and storage: URLs can be stored or reused in ways that expose sensitive values. ●

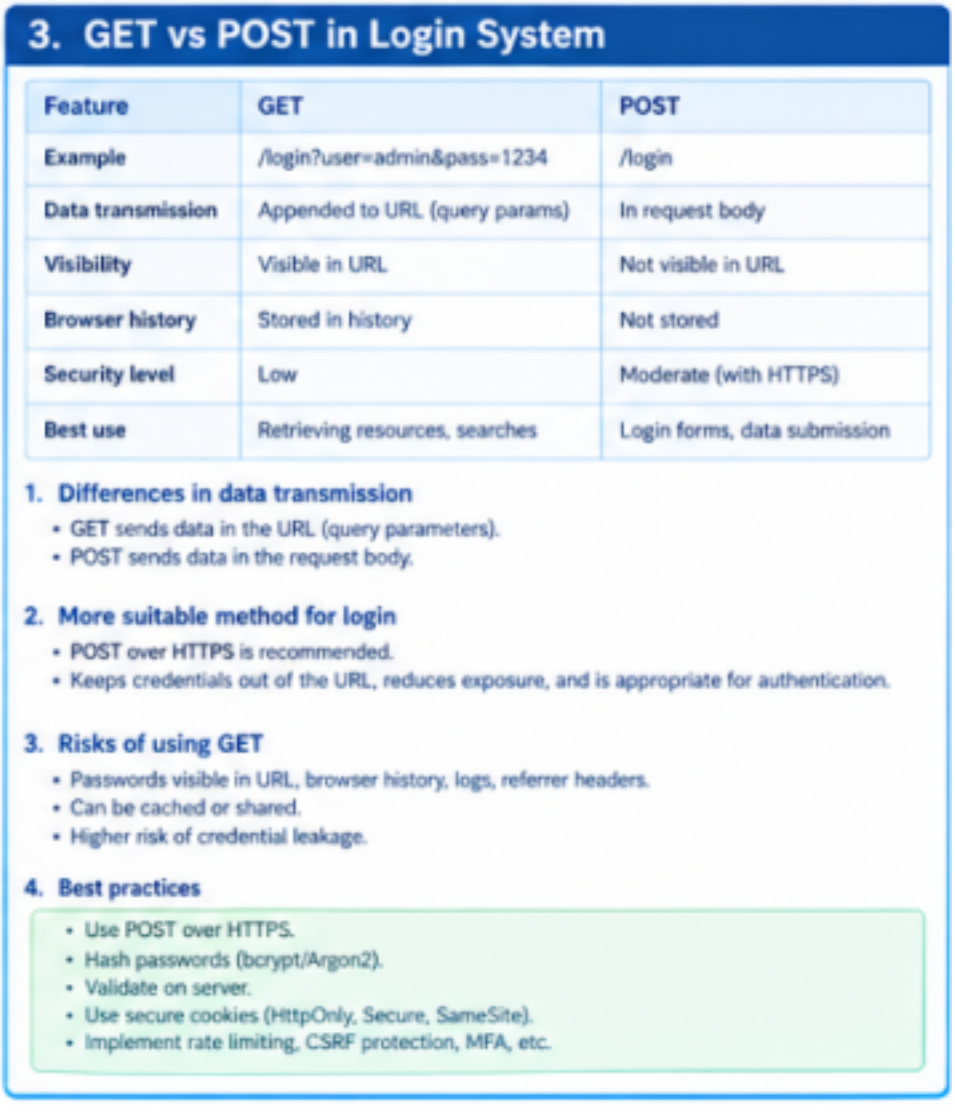
Credential disclosure: Anyone who gains access to the exposed URL may obtain the credentials.

HTTPS does not eliminate these URL-related risks. Sensitive data should not be placed in query parameters, even when HTTPS is enabled.

3.4 Suggest best practices for secure login implementation.

1. Use POST over HTTPS for login submissions.
2. Hash stored passwords with a dedicated password-hashing algorithm such as Argon2id, bcrypt, or scrypt.
3. Validate credentials on the server and return generic authentication error messages. 4. Store session identifiers in cookies configured with `Secure`, `HttpOnly`, and an appropriate `SameSite` attribute.
5. Apply rate limiting and account protection against brute-force attacks.
6. Use multi-factor authentication where appropriate.
7. Protect cookie-based authentication flows against cross-site request forgery (CSRF).
8. Never log plaintext passwords or include them in URLs.
9. Use secure session expiration and regenerate session identifiers after successful authentication.

These practices protect user credentials and reduce the risk of unauthorized access.



3. GET vs POST in Login System

Feature	GET	POST
Example	/login?user=admin&pass=1234	/login
Data transmission	Appended to URL (query params)	In request body
Visibility	Visible in URL	Not visible in URL
Browser history	Stored in history	Not stored
Security level	Low	Moderate (with HTTPS)
Best use	Retrieving resources, searches	Login forms, data submission

1. Differences in data transmission

- GET sends data in the URL (query parameters).
- POST sends data in the request body.

2. More suitable method for login

- POST over HTTPS is recommended.
- Keeps credentials out of the URL, reduces exposure, and is appropriate for authentication.

3. Risks of using GET

- Passwords visible in URL, browser history, logs, referrer headers.
- Can be cached or shared.
- Higher risk of credential leakage.

4. Best practices

- Use POST over HTTPS.
- Hash passwords (bcrypt/Argon2).
- Validate on server.
- Use secure cookies (HttpOnly, Secure, SameSite).
- Implement rate limiting, CSRF protection, MFA, etc.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors. 4. Suggest a corrected version of the code.

Given HTML snippet:

<!DOCTYPE html>

<html>

<head>

<title>Sample</title>
</head>

<body>

<h1>Welcome

<p>This is a paragraph

<img src="image.jpg">

</body>

</html>

4.1 Identify missing closing tags.

The following issues are present:

- `<h1>` is missing its explicit closing tag `</h1>`.
- `<p>` is missing its explicit closing tag `</p>`.
- The `<html>` element does not specify a language using the `lang` attribute.
- The document lacks character encoding and viewport metadata.

The `<img>` element is a void element in HTML, so it does not require a closing tag.

4.2 Analyze issues with the `<img>` tag.

The image element is:

<img src="image.jpg">

The main issues are:

1. Missing alternative text: The `alt` attribute is absent, reducing accessibility for users who rely on screen readers.
2. Missing dimensions: Explicit `width` and `height` attributes can help reserve layout space and reduce layout shifts.
3. Image availability: The image may not display if `image.jpg` does not exist at the specified path.
4. Responsive presentation: CSS can ensure the image fits different screen sizes.

For an informative image, meaningful alternative text should describe its content. For a purely decorative image, use `alt=""`.

4.3 Determine how browsers will handle these errors.

Modern browsers use error-recovery rules to interpret malformed HTML.

- The browser may automatically close the heading when it encounters the paragraph. ● The paragraph may be closed implicitly when the parser encounters another element or the end of the document.
- The image may render if the resource exists and loads successfully.
- Missing alternative text does not necessarily prevent image rendering, but it harms accessibility.
- Missing image dimensions may cause content to shift when the image loads.

Although the browser can recover, the developer should correct the source rather than rely on automatic error recovery.

4.4 Suggest a corrected version of the code.

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Sample</title>

</head>

<body>

    <main>

        <h1>Welcome</h1>
        <p>This is a paragraph.</p>



        <img src="image.jpg"

            alt="Description of the image"

            width="600"

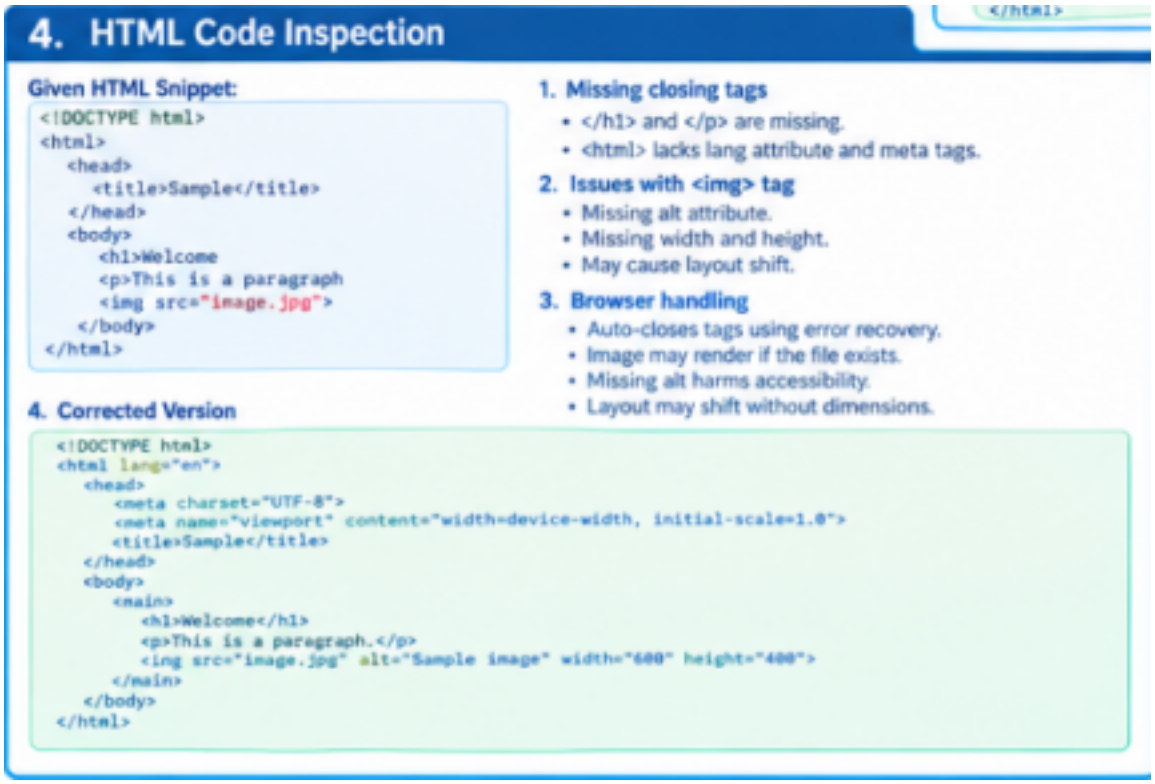
            height="400">

    </main>

</body>

</html>
```

Explanation: The corrected code explicitly closes the heading and paragraph, adds essential metadata, and provides alternative text and image dimensions. The image dimensions should match the actual image or its intended aspect ratio.



5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Browser–Server Communication Flow

The given sequence describes how a browser communicates with a web server when a user requests a webpage.

5.1 Interpret each step in the communication flow.

Step 1: User enters a URL

The user enters a website address, such as `https://example.com`, into the browser. The browser identifies the destination hostname and requested resource.

Step 2: DNS lookup

The browser or operating system resolves the hostname to an IP address using available caches or a DNS resolver. This helps identify the server to contact.

Step 3: HTTP request is sent

The browser establishes the necessary network connection and, for HTTPS, performs or resumes a secure TLS connection. It sends an HTTP request containing the method, path, headers, and possibly a request body.

Step 4: Server processes the request

The server receives the request, performs routing and application logic, accesses databases or other services if required, and prepares a response.

Step 5: Response is returned
The server sends an HTTP status code, response headers, and a response body when applicable. The response may contain HTML, JSON, images, or other resources.

Step 6: Browser renders the page

The browser parses HTML to build the DOM, processes CSS to construct the CSSOM, executes applicable JavaScript, and lays out and paints the page. Additional resources may trigger further requests.

5.2 Identify where delays can occur.

Delays can occur at several stages:

1. DNS resolution: Slow DNS servers or cache misses can increase lookup time.

2. Network connection: High latency, packet loss, or congestion can delay connection establishment. 3. TLS handshake: HTTPS setup can add latency, especially when a new connection is required. 4. Server processing: Expensive application logic, overloaded servers, or slow database queries can increase response time.

5. Data transfer: Large HTML files, images, scripts, and other resources take longer to download. 6. Browser rendering: Complex CSS, large DOM trees, heavy JavaScript, and layout recalculations can delay page display.

Network latency, server response time, and frontend processing all contribute to the user's perceived loading time.

5.3 Analyze the role of DNS in the process.

The Domain Name System (DNS) translates human-readable domain names into IP addresses that computers use to communicate.

For example:

`www.example.com` → IP address of the destination server.

DNS performs several important functions:

- Resolves hostnames into IP addresses.
- Helps browsers identify the correct destination server.
- Supports caching to reduce repeated lookup delays.
- Provides records used for services such as mail delivery and other domain-related operations. DNS

does not transfer the webpage itself. It helps the browser determine where to send its network request. **5.4**

Suggest optimizations to improve performance.

The following optimizations can reduce loading time:

1. DNS optimization: Use reliable DNS resolvers and suitable DNS caching and TTL settings. 2. Connection reuse: Use persistent connections and modern protocols such as HTTP/2 or HTTP/3 where supported.

3. HTTPS optimization: Use TLS session resumption and correctly configured secure connections. 4. Server optimization: Improve application code, database queries, caching, and server resource utilization. 5. Content delivery network (CDN): Serve static resources from geographically distributed locations. 6. Compression: Enable Brotli or gzip compression for suitable text-based resources. 7. Image optimization: Resize images appropriately and use modern formats such as WebP or AVIF where supported.

8. Frontend optimization: Minimize unnecessary JavaScript and CSS, defer non-critical scripts, and remove unused code.

9. Browser caching: Configure appropriate cache-control headers for static assets.

10. Lazy loading: Load off-screen images and other non-critical resources when needed. 11. Performance monitoring: Measure metrics such as Time to First Byte (TTFB), Largest Contentful Paint (LCP), and Interaction to Next Paint (INP) to identify bottlenecks.

These techniques reduce network overhead, server processing time, and rendering delays, improving the overall user experience.

